Operating Systems and Distributed Systems

Lab 1: Visualizing Distributed Protocols

Cristian · NTP · Lamport & vector clocks · Ricart–Agrawala

Stanley [**TODO:** full name]
[**TODO:** student ID]

October 29, 2026

Contents

1 Overview **3**
1.1 Goal of this report . . . 3
1.2 Environment . . . 3
1.3 How the evidence is presented . . . 3
1.4 Conventions . . . 3

2 Cristian clock synchronization **4**
2.1 Algorithm . . . 4
2.2 Implementation . . . 4
2.3 Scenario coverage . . . 5
2.4 Normal: one sync over symmetric links . . . 5
2.5 Fault A: the time server crashes . . . 6
2.6 Fault B: asymmetric link delay . . . 6
2.7 Handout scenario C: server queuing delay . . . 7
2.8 Design variant: +0 vs. +RTT/2 vs. +RTT . . . 8
2.9 Discussion questions . . . 8
2.10 Boundaries (what is not implemented) . . . 8

3 NTP (Network Time Protocol) **9**
3.1 Algorithm . . . 9
3.2 Implementation . . . 9
3.3 Scenario coverage . . . 10
3.4 Normal (handout A): server processing time is subtracted . . . 10
3.5 Handout B: minimum-delay filtering . . . 11
3.6 Handout C: error accumulates with stratum . . . 11
3.7 Fault A: an upstream server crashes . . . 12
3.8 Fault B: partition or asymmetric link . . . 12
3.9 Design variant: [**TODO:** name] . . . 12
3.10 Discussion questions . . . 13
3.11 Boundaries (what is not implemented) . . . 13

4 Lamport clocks and vector clocks **14**
4.1 Algorithm . . . 14
4.2 Implementation . . . 14
4.3 Scenario coverage . . . 14
4.4 Normal (handout A): the example on lecture slide 37 . . . 15
4.5 Handout B: concurrent events with equal Lamport values . . . 16
4.6 Handout C: invariant after random events . . . 16
4.7 Fault A: a node crashes and recovers . . . 16

4.8 Fault B: blocked link or high latency . . . 16
4.9 Design variant: [**TODO:** name] . . . 17
4.10 Discussion questions . . . 17
4.11 Boundaries (what is not implemented) . . . 17

5 Ricart–Agrawala mutual exclusion **18**
5.1 Algorithm . . . 18
5.2 Implementation . . . 18
5.3 Scenario coverage . . . 19
5.4 Normal (handout A): two nodes request at the same time . . . 19
5.5 Fault A (handout B): an uninvolved node crashes . . . 19
5.6 Fault B (handout C): random load on an exponentially jittered network . . . 20
5.7 Design variant: [**TODO:** name] . . . 20
5.8 Discussion questions . . . 21
5.9 Boundaries (what is not implemented) . . . 21

6 Summary **22**
6.1 Comparison of the three physical-clock / logical-clock approaches . . . 22
6.2 What each design variant taught me . . . 22
6.3 Problems I ran into . . . 22
6.4 Bonus (optional) . . . 22

A Reproduction index **23**

1 Overview

1.1 Goal of this report

This report covers my part of Lab 1 (`docs/LAB1.en.md`): four protocols, each implemented as a Go module in `protocols/` and run on the DistVis platform. For every protocol I show the normal behaviour, a crash scenario, a partition or high-latency scenario, and a design variant, as required by §2.1 of the handout.

[**TODO:** One or two sentences: which teammates cover the remaining required protocols (Berkeley, Bully, Lamport mutual exclusion, 2PC, . . .) and where their part is.]

Module	Protocol	Code directory	Handout
L03 Clocks	Cristian	`protocols/christian`	§3.1
L03 Clocks	NTP	`protocols/ntp`	§3.2
L03 Clocks	Lamport + vector	`protocols/logical-clocks` [**TODO:** or `protocols/lamport`?]	§3.4
L04 Mutual exclusion	Ricart–Agrawala	`protocols/randa`	§4.4

Table 1: Protocols covered in this report.

1.2 Environment

Platform	DistVis, started with `npm start`, UI at `http://localhost:3000`
Node.js	[**TODO:** version]
Go	[**TODO:** version]
Docker	[**TODO:** version]
Runtime mode	Docker (real Go nodes) for every result in this report, unless stated otherwise
Machine / OS	[**TODO:** CPU, OS]

Hint: The built-in simulation does not run your Go code (LAB1 §1.3). Say clearly which mode each run used.

1.3 How the evidence is presented

- **Space-time diagrams.** Screenshots from the platform. Horizontal axis is time, one row per node; solid arrow = delivered message, dashed = in flight, red × = dropped, coloured square = a `node.Report(...)` state report.
- **Exported runs.** JSON files from "Run history → Export record", stored in `report`. Each scenario names its file.
- **Automated scenarios.** Where a protocol has a `scenarios.json`, the scenario can be replayed with `node scripts/scenarios.mjs run <protocol> <scenario-id>` (`scripts/scenarios.mjs`).

1.4 Conventions

- `errorMs` = (node's simulated clock) − (real time), in milliseconds. Positive means the node is ahead. The algorithms never read it; it exists only so an observer can judge accuracy.
- Nodes are called `node-1`, . . . , `node-n`. Link delays are written `node-a→node-b = 200 ms` (one direction only unless stated).
- [**TODO:** any other convention you use, e.g. rounding of numbers]

2 Cristian clock synchronization

Handout: LAB1 §3.1
Code: `protocols/christian`
README: `protocols/christian/README.md`

2.1 Algorithm

A client reads its clock (T_0), asks the time server for the time, receives the server's reading T_s, and reads its clock again (T_1). It then sets

$$C \leftarrow T_s + \frac{T_1 - T_0}{2}, \qquad \text{worst-case error} \le \frac{T_1 - T_0}{2} = \frac{\text{RTT}}{2}.$$

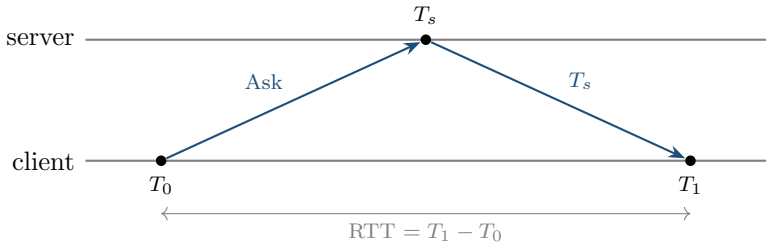


Figure 1: One Cristian round.

[**TODO:** 2–4 sentences in your own words: why the midpoint, and what assumption (symmetric delay) the formula depends on.]

2.2 Implementation

- Formula: [**TODO:** function name] in `protocols/christian/protocol.go`
- Client side (one `Sync`): [**TODO:**] Server side (`TimeService.Ask`): [**TODO:**]
- Simulated clock (offset + drift): `protocols/christian/clock.go`
- Automated scenarios: `protocols/christian/scenarios.json`

State fields reported

Field	Meaning
`role`	[**TODO:**]
`errorMs`	[**TODO:**]
`driftPPM`	[**TODO:**]
`syncs` / `failedSyncs`	[**TODO:**]
`lastSync`	[**TODO:** (t_0, T_s, t_1, RTT, estimate, correction, error bound, error before/after)]
`lastError`	[**TODO:**]

Application inputs

Input	Fields	Effect
`Application.Sync`	`server`, `mode`	[**TODO:**]
`Application.SetServerDelay`	`delayMs`	[**TODO:**]
`Application.SetClock`	`offsetMs`, `driftPPM`	[**TODO:**]

2.3 Scenario coverage

Report section	Handout	What it shows	Export
2.4	A	symmetric sync, error → 0	`cristian_scenA_normal.json`
2.5	(fault A)	time server crashes	`cristian_scenB_crash.json`
2.6	B	asymmetric link	`cristian_scenC_asymmetric.json`
2.7	C	server queuing delay	`cristian_scenD_serverDelay.json`
2.8	variant	+0 vs +RTT/2 vs +RTT	`cristian_designVariant.json`

2.4 Normal: one sync over symmetric links

Setup

Nodes / latency	3 nodes, [**TODO:** 80] ms, Docker
Scenario id	`a-normal`
Export	`report/cristian_scenA_normal.json`

Steps

1. Record the initial `errorMs` of every node.
2. On node-2 run `Application.Sync` with `server=node-1`, `mode=half`.
3. [**TODO:**]

Result

Node	errorBefore (ms)	RTT (ms)	correction (ms)	errorAfter (ms)	bound (ms)
node-2	—	—	—	—	—
node-3	—	—	—	—	—

Screenshot placeholder
put `figures/cristian-normal.png` here

Figure 2: Cristian normal case: request/reply arrows and the state report carrying `lastSync`.

Explanation

[**TODO:** 3–5 sentences: what you saw, did it match the expectation, why. Mention drift after the sync.]

2.5 Fault A: the time server crashes

Setup

Nodes / latency	3 nodes, [**TODO:**] ms, Docker
Scenario id	`b-crash`
Export	`report/cristian_scenB_crash.json`

Steps

1. Crash node-1 ("Simulate crash").
2. On node-2 run `Sync`; wait for the RPC timeout.
3. Recover node-1 and run `Sync` again.

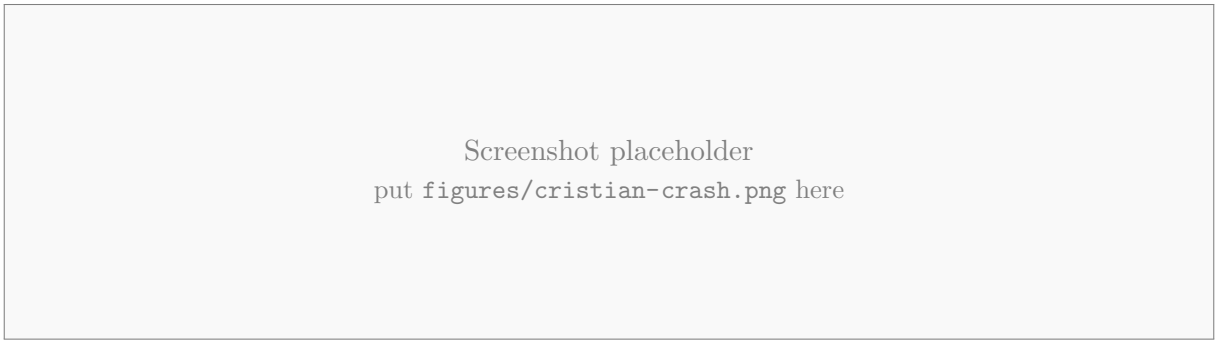


Figure 3: Request dropped (red ×) while node-1 is down.

Explanation

[**TODO:** 3–5 sentences. Was the clock changed by the failed sync? What does `lastError` say? Single point of failure.]

2.6 Fault B: asymmetric link delay

Setup

Nodes / latency	3 nodes, [**TODO:** 200] ms, Docker
Link faults	node-2→node-1 = 2000 ms, node-1→node-2 = 200 ms, then reversed
Scenario id	`c-asymmetric`
Export	`report/cristian_scenC_asymmetric.json`

Expected value

With request delay d_{req} and reply delay d_{rep}, the error after the sync is $\dfrac{d_{\text{req}} - d_{\text{rep}}}{2}$ (derive it in one line: [**TODO:**]).

Result

Links (req / reply)	RTT	assumed reply delay	predicted error	measured `errorAfterMs`
2000 / 200	___	___	+900	___
200 / 2000	___	___	−900	___
healed	___	___	0	___



Figure 4: Asymmetric link: long request arrow, short reply arrow.

Explanation

[**TODO:** 3–5 sentences. Can the client detect the asymmetry from the RTT alone?]

2.7 Handout scenario C: server queuing delay

Setup

Nodes / latency	3 nodes, [**TODO:**] ms, Docker
Fault	`SetServerDelay(1000)` on node-1
Scenario id	`d-server-delay`
Export	`report/cristian_scenD_serverDelay.json`

Result

Server delay	RTT (ms)	predicted error (ms)	measured error (ms)
1000 ms	—	—	—
0 ms	—	—	—

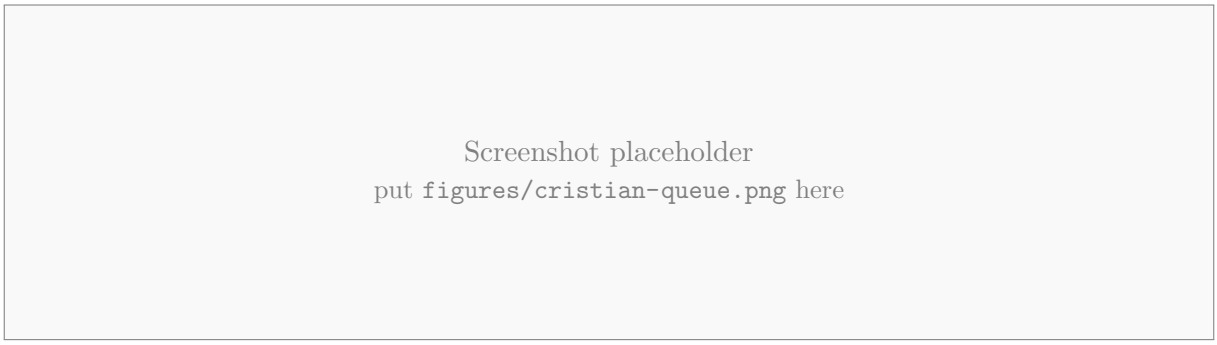


Figure 5: Server queuing delay: gap between request arrival and reply on node-1.

Explanation

[**TODO:** 3–5 sentences. Why is the server's waiting time mistaken for network time, and how does NTP's $t_2 - t_1$ fix it (link to Section 3).]

2.8 Design variant: +0 vs. +RTT/2 vs. +RTT

Setup

Nodes / latency	3 nodes, [**TODO:** 200] ms, Docker
Variant switch	`mode` field of `Sync`: `zero`, `half`, `full`
Scenario id	`e-variant`
Export	`report/cristian_designVariant.json`

Result

Mode	Estimate	RTT	measured error	worst-case bound
`zero`	T_s	—	—	—
`half`	$T_s + \mathrm{RTT}/2$	—	—	—
`full`	$T_s + \mathrm{RTT}$	—	—	—

Explanation

[**TODO:** 3–5 sentences: why the midpoint minimises the worst-case error.]

2.9 Discussion questions

How large is the error when the delays are not symmetric?

[**TODO:** 2–4 sentences.]

Why add RTT/2 rather than 0 or RTT?

[**TODO:** 2–4 sentences.]

[TODO: other question from the protocol README]

[**TODO:** 2–4 sentences.]

2.10 Boundaries (what is not implemented)

- [**TODO:** e.g. no periodic sync, no server failover, no multi-sample filtering, corrections are steps (can move the clock backwards), clocks not persisted.]

3 NTP (Network Time Protocol)

Handout: LAB1 §3.2
Code: `protocols/ntp`
README: `protocols/ntp/README.md`

3.1 Algorithm

Four timestamps per exchange: t_0 client send, t_1 server receive, t_2 server send, t_3 client receive.

$$\textit{offset} = \frac{(t_1 - t_0) + (t_2 - t_3)}{2}, \qquad \textit{delay} = (t_3 - t_0) - (t_2 - t_1).$$

Out of several samples, the client uses the *offset* of the sample with the *smallest delay*. A node's `stratum` is its upstream's stratum +1.

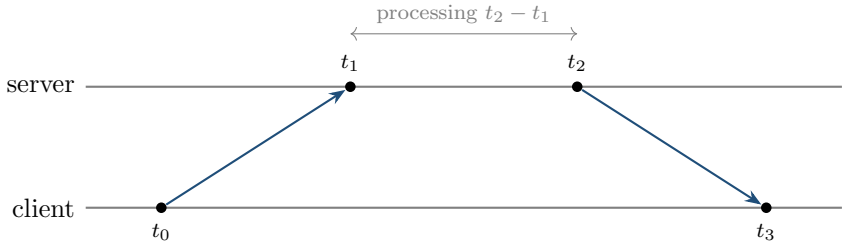


Figure 6: One NTP exchange. The processing interval is removed from *delay*.

[**TODO:** 2–4 sentences in your own words: how the four timestamps improve on Cristian (Section 2.7) and what they still cannot fix (asymmetry).]

3.2 Implementation

- Offset/delay formulas: [**TODO:** function] in `protocols/ntp/protocol.go`
- Minimum-delay filter: [**TODO:** function]
- Stratum / upstream selection: [**TODO:** function]
- RPC wiring: `protocols/ntp/rpc.go`; clock model: `protocols/ntp/clock.go`
- Tests: `protocols/ntp/protocol_test.go`

State fields reported

Field	Meaning
`stratum` / `upstream`	[**TODO:**]
`last.samples`	[**TODO:** (per sample: $t_0..t_3$, delay, offset, error)]
`last.selected`	[**TODO:** (zero-based index; -1 = no valid sample)]
`lastError`	[**TODO:**]
[**TODO:** more]	

Application inputs

Input	Fields	Effect
`Application.Sync`	`servers`, `samples`, `processingMS`	[TODO:]
`Application.ConfigureClock`	`offsetMS`, `driftPPM`	[TODO:]
`Application.Read`	—	[TODO:]

3.3 Scenario coverage

Report section	Handout	What it shows	Export
3.4	A	processing time $t_2 - t_1$ is subtracted	[TODO: file]
3.5	B	minimum-delay filtering (8 samples, jitter)	[TODO: file]
3.6	C	error accumulates with stratum	[TODO: file]
3.7	(fault A)	upstream crashes	[TODO: file]
3.8	(fault B)	partition / asymmetric link	[TODO: file]
3.9	variant	[TODO: your choice]	[TODO: file]

3.4 Normal (handout A): server processing time is subtracted

Setup

Nodes / latency	3 nodes, [TODO: 80] ms, Docker
Input	node-2: `Sync servers=["node-1"], samples=___, processingMS=200`
Export	[TODO:]

Steps

1. [TODO:]

Result (one sample)

t_0	t_1	t_2	t_3	$t_2 - t_1$	delay	offset
___	___	___	___	___	___	___

Screenshot placeholder
put `figures/ntp-normal.png` here

Figure 7: NTP exchange; the gap on node-1 between receive and reply is the processing time.

Explanation

[TODO: 3–5 sentences. Compare the measured send/receive times in the diagram with $t_0..t_3$; show $t_2 - t_1 \approx$ processingMS and that *delay* excludes it.]

3.5 Handout B: minimum-delay filtering

Setup

Nodes / latency	3 nodes, [**TODO:**] ms with jitter [**TODO:** distribution / amount]
Input	`Sync` with `samples=8`
Export	[**TODO:**]

Result

#	delay (ms)	offset (ms)	selected?
0	—	—	
1	—	—	
	[**TODO:** . . . up to 7]		

Screenshot placeholder
put `figures/ntp-filter.png` here

Figure 8: Eight samples under jitter.

Explanation

[**TODO:** 3–5 sentences. Why pick the smallest delay and not the smallest offset?]

3.6 Handout C: error accumulates with stratum

Setup

Nodes / latency	3 nodes, [**TODO:**] ms
Chain	node-1 (stratum 1) → node-2 → node-3
Export	[**TODO:**]

Steps

1. On node-3, try `Sync` from node-2 *before* node-2 is synchronised (expect failure).
2. Sync node-2 from node-1, then node-3 from node-2.
3. [**TODO:**]

Result

Node	stratum	upstream	final `errorMs`
node-1	1	—	—
node-2	—	node-1	—
node-3	—	node-2	—

Explanation

[**TODO:** 3–5 sentences.]

3.7 Fault A: an upstream server crashes

Setup and steps

1. [**TODO:** e.g. crash node-1 and sync node-2 from node-1 (it fails). Then sync node-3 from both node-1 and node-2 and show that node-2 still supplies a sample.]

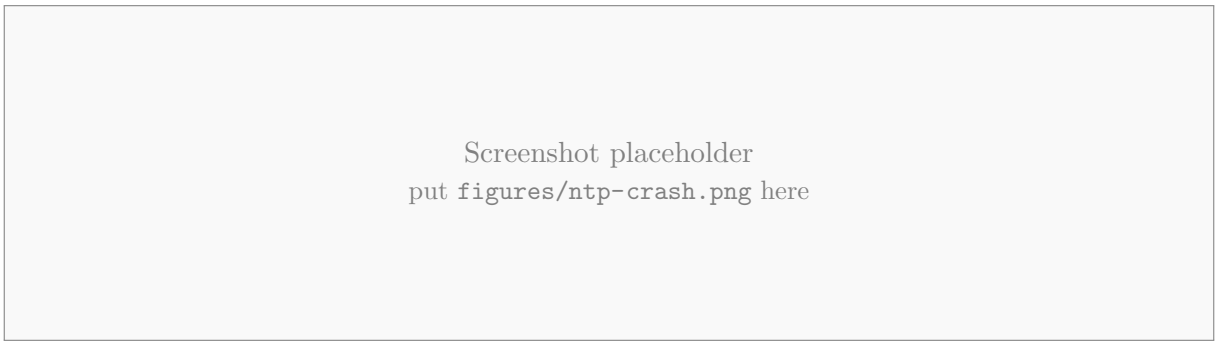


Figure 9: [**TODO:** caption]

Explanation

[**TODO:** 3–5 sentences. Is the clock left unchanged on failure? What does `selected` show?]

3.8 Fault B: partition or asymmetric link

Setup and steps

1. [**TODO:** e.g. set node-2→node-1 = 2000 ms, node-1→node-2 = 200 ms and sync; compare with Cristian's error in Section 2.6.]

Result

[**TODO:** table: predicted bias $(d_{\text{fwd}} - d_{\text{rev}})/2$ vs. measured]

Explanation

[**TODO:** 3–5 sentences. Four timestamps remove server work, but not asymmetric network delay.]

3.9 Design variant: [TODO: name]

Hint: The handout gives no fixed NTP variant. Ideas: (1) choose the sample with the smallest —offset— instead of the smallest delay; (2) do not subtract $t_2 - t_1$ (i.e. fall back to Cristian); (3) average all offsets. Pick one, describe the code change, and show the error it causes.

Code change

[**TODO:** what you changed, with a link to the line]

Result

[**TODO:** baseline vs. variant, same scenario]

Explanation

[**TODO:** 3–5 sentences: why the original design is needed.]

3.10 Discussion questions

Why pick the smallest delay rather than the smallest offset?

[**TODO:** 2–4 sentences.]

Why does error accumulate with stratum?

[**TODO:** 2–4 sentences.]

[TODO: other question]

[**TODO:** 2–4 sentences.]

3.11 Boundaries (what is not implemented)

- [**TODO:** e.g. no NTP packet format/UDP, no periodic polling, no falseticker rejection, steps instead of slewing, no external UTC.]

4 Lamport clocks and vector clocks

Handout: LAB1 §3.4
Code: [**TODO:** `protocols/logical-clocks` or `protocols/lamport`]

4.1 Algorithm

Every node i keeps a scalar L and a vector V of length n.

Event at node i	Lamport	Vector
local or send	$L \leftarrow L + 1$	$V[i] \leftarrow V[i] + 1$
receive (L_m, V_m)	$L \leftarrow \max(L, L_m) + 1$	$V[k] \leftarrow \max(V[k], V_m[k])\ \forall k$, then $V[i] \leftarrow V[i] + 1$

Total-order timestamp: [**TODO:** $L.i$, or $n \cdot L + i$ – write the one your code uses].

Properties to verify:

- $e \to e' \Rightarrow L(e) < L(e')$, but not the converse.
- $V(e) < V(e') \iff e \to e'$; otherwise the events are concurrent ($e \parallel e'$).
- The total order is arbitrary for concurrent events.

4.2 Implementation

- Clock update rules: [**TODO:** function names + links]
- Message piggybacking (what is sent with each message): [**TODO:**]
- `Compare`: [**TODO:** where, and what it returns]
- Concurrency: [**TODO:** which mutex protects the clock; see the variant in Section 4.9]

State fields and inputs

Field / input	Meaning
[**TODO:** `lamport`]	[**TODO:**]
[**TODO:** `vector`]	[**TODO:**]
[**TODO:** `events`]	[**TODO:**]
[**TODO:** `Local`]	[**TODO:**]
[**TODO:** `Send`]	[**TODO:**]
[**TODO:** `Compare`]	[**TODO:**]
[**TODO:** `Auto`]	[**TODO:**]

4.3 Scenario coverage

Report section	Handout	What it shows	Export
4.4	A	lecture slide 37: $L(b) > L(e)$ but $b \parallel e$	[**TODO:** file]
4.5	B	three concurrent events with equal L	[**TODO:** file]
4.6	C	invariant over random events (`Auto`)	[**TODO:** file]
4.7	(fault A)	node crash and recovery	[**TODO:** file]
4.8	(fault B)	blocked link: send without receive	[**TODO:** file]
4.9	variant	[**TODO:**]	[**TODO:** file]

4.4 Normal (handout A): the example on lecture slide 37

Setup

Nodes / latency	3 nodes, [**TODO:**] ms, Docker
Export	[**TODO:**]

Steps

1. [**TODO:** the exact sequence of Local / Send inputs that reproduces the figure]

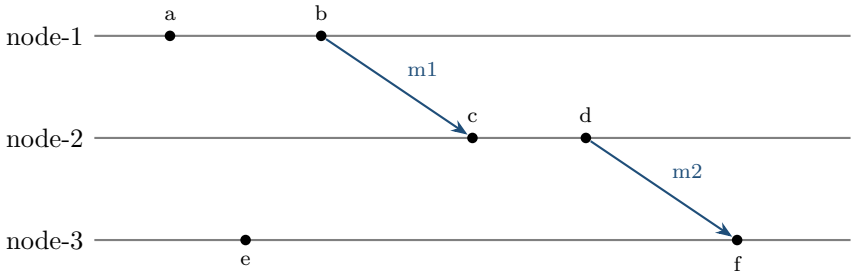


Figure 10: Expected event structure ([**TODO:** check against slide 37]).

Result

Event	Node	Kind	L	V	total order
a	node-1	local	___	___	___
b	node-1	send	___	___	___
c	node-2	receive	___	___	___
d	node-2	send	___	___	___
e	node-3	local	___	___	___
f	node-3	receive	___	___	___

`Compare` results

Pair	Lamport says	Vector says	Agree?
(b, e)	___	___	___
(b, f)	___	___	___
[**TODO:**]			

Screenshot placeholder
put `figures/lc-slide37.png` here

Figure 11: Space-time diagram reproducing slide 37.

Explanation

[**TODO:** 3–5 sentences. Why $L(b) > L(e)$ does not mean $e \to b$.]

4.5 Handout B: concurrent events with equal Lamport values

Steps

1. On a fresh run, have node-1, node-2, node-3 each produce one local event (no messages).
2. `Compare` each pair.

Result

[**TODO:** table: three events, $L = 1$, vectors, total-order numbers, pairwise comparison]

Explanation

[**TODO:** 3–5 sentences. Does the order $1.1 < 1.2 < 1.3$ have any physical meaning?]

4.6 Handout C: invariant after random events

Setup

Nodes / latency	[**TODO:**]
Auto settings	[**TODO:** rate, duration]
Export	[**TODO:**]

Result

Event 1	Event 2	Lamport order	Vector relation	`agrees`
—	—	—	—	—
—	—	—	—	—
—	—	—	—	—

Explanation

[**TODO:** 3–5 sentences. Every causal pair agrees; some Lamport-ordered pairs are concurrent.]

4.7 Fault A: a node crashes and recovers

Steps

1. [**TODO:** produce events, crash node-2, send to it, recover, produce more events]

Explanation

[**TODO:** 3–5 sentences. Are the counters restored (`sdk.Save`/`sdk.Load`)? What would go wrong if a recovered node restarted at $L = 0$?]

4.8 Fault B: blocked link or high latency

Steps

1. [**TODO:** block node-1→node-2, Send, heal, Send again]

Explanation

[**TODO:** 3–5 sentences. A send event exists without a receive event; the vector clocks still describe causality correctly.]

4.9 Design variant: [TODO: name]

Hint: The handout gives no fixed variant here. Options: (1) on receive, skip the max *and just increment – then $e \to e'$ no longer implies $L(e) < L(e')$; (2) merge vectors without* max *(copy the incoming vector) – causality is lost; (3) remove the mutex around the clock (your* `protocols/lamport` *code has this experiment commented in) and show the corrupted state under concurrent sends.*

Code change

[**TODO:**]

Result

[**TODO:** baseline vs. variant]

Explanation

[**TODO:** 3–5 sentences.]

4.10 Discussion questions

Why does $L(e) < L(e')$ not imply $e \to e'$?

[**TODO:** 2–4 sentences.]

Does the total order among concurrent events have physical meaning?

[**TODO:** 2–4 sentences.]

What does a vector clock cost compared with a Lamport clock?

[**TODO:** 2–4 sentences (message size $O(n)$, fixed membership).]

4.11 Boundaries (what is not implemented)

- [**TODO:** e.g. fixed membership, no causal-delivery buffer, no total-order broadcast.]

5 Ricart–Agrawala mutual exclusion

Handout: LAB1 §4.4
Code: protocols/randa

5.1 Algorithm

To enter the critical section (CS), node i stamps a request with its Lamport timestamp (L, i) and sends `Request` to all $n-1$ other nodes. It enters when it has received a `Reply` from all of them. A node receiving a request:

- replies at once if it is neither holding nor wanting the CS, or if it wants the CS but the incoming timestamp is smaller (higher priority);
- otherwise *defers* the reply until it leaves the CS.

There is no `Release` message: the deferred replies act as the release. Messages per CS entry: $2(n-1)$ (Lamport mutual exclusion needs $3(n-1)$).

[**TODO:** 2–4 sentences in your own words: why the timestamp total order prevents deadlock and guarantees mutual exclusion.]

5.2 Implementation

- Lock state machine (indifferent / waiting / holding): `ralock` in protocols/randa/locks.go
- Handling an incoming request (the "defer" decision): `ralock.question` in protocols/randa/locks.go
- Broadcasting requests and waiting for all replies: `ralock.lock`
- RPC handlers: protocols/randa/application.go; Lamport clock: protocols/randa/clock.go
- [**TODO:** local lock / how concurrent Acquire on the same node is handled]

State fields reported

Hint: LAB1 §4 asks all mutual-exclusion protocols to report `role`, `entries`, `lastWaitMs`, `messagesSent`. Make sure randa reports these.

Field	Meaning
`role`	[**TODO:** (released / wanted / critical)]
`entries`	[**TODO:**]
`lastWaitMs`	[**TODO:**]
`messagesSent`	[**TODO:**]
`deferred`	[**TODO:**]
`clock`	[**TODO:**]

Application inputs

Input	Fields	Effect
[**TODO:** `Application.Acquire`]	`holdMs`	[**TODO:**]
[**TODO:** `Application.Auto`]	[**TODO:**]	[**TODO:**]
`Application.Lock` / `Unlock`	—	[**TODO:**]

5.3 Scenario coverage

Report section	Handout	What it shows	Export
5.4	A	two simultaneous requests	[**TODO:** file]
5.5	B	an uninvolved node crashes, everyone blocks	[**TODO:** file]
5.6	C	random load, exponential jitter, FIFO vs. unordered	[**TODO:** file]
5.7	variant	[**TODO:**]	[**TODO:** file]

5.4 Normal (handout A): two nodes request at the same time

Setup

Nodes / latency	3 nodes, [**TODO:**] ms, Docker
Input	node-1 and node-3: `Acquire(holdMs=___)` concurrently
Export	[**TODO:**]

Steps

1. [**TODO:**]

Result

Node	request timestamp	entered CS at	`lastWaitMs`	`messagesSent`
node-1	___	___	___	___
node-3	___	___	___	___

Screenshot placeholder
put `figures/ra-normal.png` here

Figure 12: Crossing requests; the higher-priority node replies immediately, the other defers.

Explanation

[**TODO:** 3–5 sentences. Point to the deferred reply in the diagram. Count messages: is it $2(n-1)$ per entry?]

5.5 Fault A (handout B): an uninvolved node crashes

Steps

1. Crash node-2 while it is not requesting the CS.
2. node-1 calls `Acquire`.
3. [**TODO:** recover node-2 and show what happens]

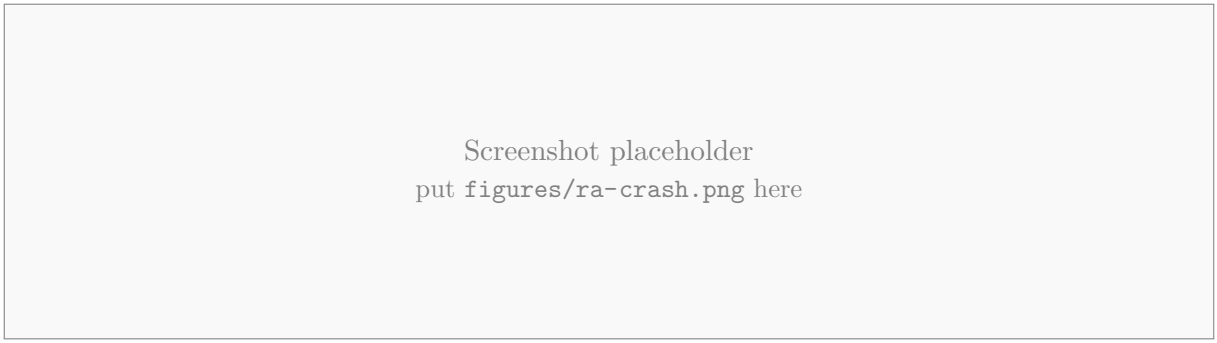


Figure 13: node-1's request to node-2 is dropped; node-1 never enters.

Explanation

[**TODO:** 3–5 sentences. Why does one crash block everyone? Compare with Lamport mutual exclusion (same weakness).]

5.6 Fault B (handout C): random load on an exponentially jittered network

Setup

Nodes / latency	5 nodes, exponential latency, mean [**TODO:**] ms
Load	`Auto` on all nodes, [**TODO:** rate / duration]
Ordering	run 1: FIFO (default); run 2: `"ordering": "unordered"`
Export	[**TODO:**]

Check

`atMostOne` on `role == critical` over the whole history (`scripts/scenarios.mjs`): [**TODO:** pass/fail for each run].

Result

Ordering	CS entries	mean wait (ms)	messages / entry	mutual exclusion held?
FIFO	—	—	—	—
unordered	—	—	—	—

Explanation

[**TODO:** 3–5 sentences. Does reordering break mutual exclusion? Why / why not for *this* implementation (note: RPC request and reply are paired).]

5.7 Design variant: [TODO: name]

Hint: Handout §4.3's variant is the message-count comparison: run the same scenario on Lamport mutual exclusion (`protocols/lamport_mutex`) and on R–A and compare `messagesSent` ($3(n-1)$ vs. $2(n-1)$). A stronger variant that shows why *a rule is needed: (1) do not defer – reply immediately even while holding the CS – and show two nodes in `role=critical`; or (2) break ties by something other than (timestamp, node id) and show a deadlock.*

Code change

[**TODO:**]

Result

Version	n	messages / CS entry	mutual exclusion held?
Ricart–Agrawala (baseline)	___	___	___
[**TODO:** variant or Lamport ME]	___	___	___

Explanation

[**TODO:** 3–5 sentences.]

5.8 Discussion questions

Why does Ricart–Agrawala need only $2(n-1)$ messages while Lamport needs $3(n-1)$?

[**TODO:** 2–4 sentences.]

Why does a crash of an uninvolved node block everyone?

[**TODO:** 2–4 sentences.]

Does message reordering break mutual exclusion?

[**TODO:** 2–4 sentences.]

5.9 Boundaries (what is not implemented)

- [**TODO:** e.g. no failure detection, no persistence, fixed membership.]

6 Summary

6.1 Comparison of the three physical-clock / logical-clock approaches

	Cristian	NTP	Lamport / vector
What it gives	[TODO:]	[TODO:]	[TODO:]
Messages per sync	[TODO:]	[TODO:]	—
Main error source	[TODO:]	[TODO:]	—
Effect of a crash	[TODO:]	[TODO:]	[TODO:]

6.2 What each design variant taught me

- **Cristian:** [TODO: one sentence]
- **NTP:** [TODO: one sentence]
- **Lamport / vector:** [TODO: one sentence]
- **Ricart–Agrawala:** [TODO: one sentence]

6.3 Problems I ran into

[TODO: Bugs you found while implementing (e.g. the race without the mutex in the Lamport code, the channel-swap race noted in `randa/locks.go`), and how you fixed them.]

6.4 Bonus (optional)

[TODO: Automated scenarios, tests, tooling, or delete this subsection.]

A Reproduction index

Every scenario in this report, with where its evidence lives. Automated scenarios run with `node scripts/scenarios.mjs run <protocol> <id>`. Exported runs are in `report`; screenshots are in `report/stanleyreport/figures/`.

Protocol	Scenario	`scenarios.json` id	Exported run (`report/`)
Cristian	normal	`a-normal`	`cristian_scenA_normal.json`
Cristian	server crash	`b-crash`	`cristian_scenB_crash.json`
Cristian	asymmetric	`c-asymmetric`	`cristian_scenC_asymmetric.json`
Cristian	server delay	`d-server-delay`	`cristian_scenD_serverDelay.json`
Cristian	variant	`e-variant`	`cristian_designVariant.json`
NTP	processing time	**[TODO:]**	**[TODO:]**
NTP	filtering	**[TODO:]**	**[TODO:]**
NTP	stratum chain	**[TODO:]**	**[TODO:]**
NTP	crash	**[TODO:]**	**[TODO:]**
NTP	partition	**[TODO:]**	**[TODO:]**
NTP	variant	**[TODO:]**	**[TODO:]**
Lamport/vector	slide 37	**[TODO:]**	**[TODO:]**
Lamport/vector	equal L	**[TODO:]**	**[TODO:]**
Lamport/vector	Auto	**[TODO:]**	**[TODO:]**
Lamport/vector	crash	**[TODO:]**	**[TODO:]**
Lamport/vector	blocked link	**[TODO:]**	**[TODO:]**
Lamport/vector	variant	**[TODO:]**	**[TODO:]**
R–A	two requests	**[TODO:]**	**[TODO:]**
R–A	crash	**[TODO:]**	**[TODO:]**
R–A	jitter	**[TODO:]**	**[TODO:]**
R–A	variant	**[TODO:]**	**[TODO:]**